What is Angular?

Angular is a **TypeScript-based** front-end framework developed by Google. It is used to build **single-page applications (SPAs)** that load dynamically without refreshing the page.

Setting Up Angular 19

Install Node.js

```
node -v
```

Install Angular CLI

```
npm install -g @angular/cli
```

Verify the installation:

```
ng version
```

Create a New Angular Project

```
ng new my-angular-app
```

Run Your Angular App

```
cd my-angular-app
ng serve
```

Angular Project Structure

Key Files & Folders in an Angular Project

📌 **src/** (Source folder)

- **app/** → Contains the main application code.
- **assets/** → Stores static assets like images, fonts, etc.
- **environments/** → Configuration files for different environments (development, production).

📌 **angular.json** → Angular project configuration (styles, scripts, build settings).
📌 **package.json** → Manages dependencies and scripts (like ng serve).
📌 **tsconfig.json** → TypeScript configuration file.

Exploring the app/ Folder

Inside src/app/, you'll find these important files:

📌 **app.module.ts** → The root module of your Angular app.
📌 **app.component.ts** → The main component logic (TypeScript).
📌 **app.component.html** → The main template (HTML).
📌 **app.component.css** → The styles for this component (CSS).
📌 **app.component.spec.ts** → Test file for unit testing.

Creating a New Component

```
ng generate component my-first-component
```

or simply:

```
ng g c my-first-component
```

This will create a new folder src/app/my-first-component/ with the following files:

- **my-first-component.component.ts** → Component logic (TypeScript)
- **my-first-component.component.html** → Component template (HTML)
- **my-first-component.component.css** → Component styles (CSS)
- **my-first-component.component.spec.ts** → Unit test file

Wherever you want to show this component you have to use a tag like this in HTML.

```
<app-my-first-component></app-my-first-component>
```

Event Binding in Angular

Event binding allows users to interact with the application by listening to **user actions** like clicks, inputs, and key presses.

1️⃣ Handling Click Events

Modify **my-first-component.component.html**:

```
<h2>{{ message }}</h2>
<button (click)="updateMessage()">Click Me</button>
```

Modify **my-first-component.component.ts**

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-my-first-component',
```

```
  templateUrl: './my-first-component.component.html',
  styleUrls: ['./my-first-component.component.css']
})
export class MyFirstComponent {
  message = 'Hello, Angular 19!';

  updateMessage() {
    this.message = 'You clicked the button!';
  }
}
```

Here, (click) is an event binding that calls updateMessage() when the button is clicked.

2️⃣ Handling Input Events

Modify **my-first-component.component.html**:

```
<h2>{{ message }}</h2>
<input type="text" (input)="updateUserMessage($event)" placeholder="Type something..." />
<p>You typed: {{ userMessage }}</p>
```

Modify **my-first-component.component.ts**

```
export class MyFirstComponent {
  message = 'Hello, Angular 19!';
  userMessage = '';

  updateUserMessage(event: any) {
    this.userMessage = event.target.value;
  }
}
```

Two-Way Data Binding with ngModel

So far, we've used event binding to update variables manually. But Angular provides two-way data binding using the ngModel directive, which automatically syncs the UI with the component logic.

What is Two-Way Data Binding?

Two-way data binding means that when the user updates the UI, the data in the component updates automatically—and vice versa.

To use ngModel, you need to import FormsModule in app.module.ts

Modify my-first-component.component.html

```
<h2>Two-Way Data Binding Example</h2>
<input type="text" [(ngModel)]="userMessage" placeholder="Type something..." />
<p>You typed: {{ userMessage }}</p>
```

[(ngModel)]="userMessage" binds the input field **both ways**—when the user types, userMessage updates, and if userMessage is changed in the code, the input updates too!

Modify my-first-component.component.ts

```
export class MyFirstComponent {
  userMessage = 'Angular is awesome!';
}
```

when you type in the input field, the changes are reflected immediately in userMessage. 🎉

Key Benefits of ngModel

✅ Automatically syncs data between UI and logic
✅ No need for manual event handling
✅ Reduces boilerplate code

Directives in Angular

Directives are special instructions in Angular that change the behavior of elements in the DOM. They are categorized into **three types**:

1. **Structural Directives** (*ngIf, *ngFor, *ngSwitch) – Modify the DOM structure
2. **Attribute Directives** (ngClass, ngStyle, custom directives) – Change element appearance/behavior
3. **Custom Directives** – User-defined directives for advanced control

1️⃣ Structural Directives

These directives **add or remove elements dynamically**.

📌 *ngIf – Conditional Rendering

Use *ngIf to **show or hide** elements based on a condition.

Modify my-first-component.component.html:

```
<h2>ngIf Example</h2>
<button (click)="toggleMessage()">Toggle Message</button>
<p *ngIf="showMessage">Hello, this is a conditional message!</p>
```

Modify my-first-component.component.ts

```
export class MyFirstComponent {
  showMessage = true;

  toggleMessage() {
    this.showMessage = !this.showMessage;
  }
}
```

When the user clicks the button, the message appears/disappears dynamically.

📌 *ngFor – Looping Over Lists

Modify my-first-component.component.html:

```
<h2>ngFor Example</h2>
<ul>
  <li *ngFor="let item of items">{{ item }}</li>
</ul>
```

Modify my-first-component.component.ts

```
export class MyFirstComponent {
  items = ['Angular', 'React', 'Vue', 'Svelte'];
}
```

This will display a list of items dynamically.

📌 *ngSwitch – Multiple Conditions

Use *ngSwitch to conditionally display elements based on a value.

Modify my-first-component.component.html:

```
<h2>ngSwitch Example</h2>
<select [(ngModel)]="selectedOption">
  <option value="angular">Angular</option>
  <option value="react">React</option>
  <option value="vue">Vue</option>
</select>

<p [ngSwitch]="selectedOption">
  <span *ngSwitchCase="'angular'">You selected Angular!</span>
  <span *ngSwitchCase="'react'">You selected React!</span>
  <span *ngSwitchCase="'vue'">You selected Vue!</span>
```

```
  <span *ngSwitchDefault>Select a framework.</span>
</p>
```

Modify my-first-component.component.ts

```
export class MyFirstComponent {
  selectedOption = '';
}
```

✅ The message changes dynamically based on the selected value.

2️⃣ Attribute Directives

These **change the appearance or behavior** of elements.

📌 ngClass – Dynamic Classes

Use ngClass to apply classes dynamically.

Modify my-first-component.component.html:

```
<h2 [ngClass]="{ 'active': isActive, 'inactive': !isActive }">ngClass Example</h2>
<button (click)="toggleActive()">Toggle Active</button>
```

Modify my-first-component.component.ts

```
export class MyFirstComponent {
  isActive = false;

  toggleActive() {
    this.isActive = !this.isActive;
  }
}
```

Modify my-first-component.component.css

```
.active { color: green; font-weight: bold; }
.inactive { color: red; font-weight: normal; }
```

📌 ngStyle – Dynamic Styles

Use ngStyle to apply styles dynamically.

Modify my-first-component.component.html

```
<h2 [ngStyle]="{ 'color': textColor, 'font-size': fontSize + 'px' }">ngStyle Example</h2>
```

```
<button (click)="increaseFont()">Increase Font</button>
```

Modify my-first-component.component.ts

```
export class MyFirstComponent {
  textColor = 'blue';
  fontSize = 16;

  increaseFont() {
    this.fontSize += 2;
  }
}
```

Clicking the button increases the font size dynamically.

3️⃣ Custom Directives

You can also create your **own directive** for custom behaviors.

📌 Creating a Custom Directive

```
ng generate directive highlight
```

or simply:

```
ng g d highlight
```

Modify highlight.directive.ts

```
import { Directive, ElementRef, HostListener } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  constructor(private el: ElementRef) {}

  @HostListener('mouseenter') onMouseEnter() {
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.el.nativeElement.style.backgroundColor = 'transparent';
  }
}
```

Modify my-first-component.component.html

```
<p appHighlight>Hover over this text to highlight it!</p>
```

Pipes in Angular

What Are Pipes?

Pipes are **built-in functions** in Angular that transform **data in the template** before displaying it.

For example, you can use a pipe to:

- Convert text to **uppercase/lowercase**
- Format **dates, currency, and numbers**
- Filter or transform **lists**
- Create **custom pipes** for advanced transformations

Built-in Pipes in Angular

Pipe Name	Usage
`uppercase`	Converts text to uppercase
`lowercase`	Converts text to lowercase
`titlecase`	Capitalizes the first letter of each word
`date`	Formats dates
`currency`	Formats currency
`percent`	Converts numbers to percentage format
`json`	Converts objects to JSON format
`slice`	Extracts a portion of an array or string

Using Pipes in Templates

Modify **my-first-component.component.html**:

```
<h2>Pipes Example</h2>
<p>Original: {{ message }}</p>
<p>Uppercase: {{ message | uppercase }}</p>
<p>Lowercase: {{ message | lowercase }}</p>
```

Modify **my-first-component.component.ts**

```
export class MyFirstComponent {
  message = 'Angular is Awesome!';
}
```

The text will be transformed automatically.

Date Pipe

```
<p>Today's Date: {{ today | date:'fullDate' }}</p>
<p>Short Date: {{ today | date:'short' }}</p>
```

Modify **my-first-component.component.ts**:

```
export class MyFirstComponent {
  today = new Date();
}
```

This format dates in different styles.

Currency & Percent Pipes

```
<p>Price: {{ price | currency:'CAD' }}</p>
<p>Discount: {{ discount | percent }}</p>
```

Modify **my-first-component.component.ts**:

```
export class MyFirstComponent {
  price = 1200;
  discount = 0.15;
}
```

This will format currency in CAD and convert 0.15 to 15%.

JSON Pipe

```
<p>Data: {{ user | json }}</p>
```

Modify **my-first-component.component.ts**:

```
export class MyFirstComponent {
  user = { name: 'Rohit shukla', role: 'Software Developer' };
}
```

Using Pipes with Parameters

```
<p>Custom Date: {{ today | date:'dd/MM/yyyy' }}</p>
<p>Rounded Price: {{ price | number:'1.2-2' }}</p>
```

Custom Pipes

You can create **your own pipe** for custom transformations.

```
ng generate pipe customTitle
```

or simply:

```
ng g p customTitle
```

Modify **custom-title.pipe.ts**:

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'customTitle'
})
export class CustomTitlePipe implements PipeTransform {
  transform(value: string): string {
    return value.charAt(0).toUpperCase() + value.slice(1).toLowerCase();
  }
}
```

Modify **my-first-component.component.html**

```
<p>Custom Pipe: {{ 'ANGULAR 19' | customTitle }}</p>
```

It converts "ANGULAR 19" to "Angular 19".

Angular Routing and Navigation

What is Routing?

Routing in Angular helps navigate between different **pages** without reloading the page.

Setting Up Routing in an Angular App

When you created your Angular project, if you **enabled routing** (Would you like to add Angular routing? → **Yes**), then the app-routing.module.ts file was already created.

If not, generate a routing module manually using:

```
ng generate module app-routing --flat --module=app
```

or simply:

```
ng g m app-routing --flat --module=app
```

Defining Routes

Open **app-routing.module.ts** and define some routes:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { AboutComponent } from './about/about.component';

const routes: Routes = [
  { path: '', component: HomeComponent }, // Default route
  { path: 'about', component: AboutComponent }, // About page
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

📌 **path**: Defines the URL path.

📌 **component**: Specifies which component to load for the given path.

📌 **RouterModule.forRoot(routes)**: Registers the routes at the root level.

Creating Components for Routing

```
ng generate component home
ng generate component about
```

Modify **home.component.html**:

```
<h2>Welcome to the Home Page</h2>
```

Modify **about.component.html**:

```
<h2>About Us</h2>
```

Adding Navigation Links

To navigate between pages, modify **app.component.html**:

```
<nav>
  <a routerLink="/">Home</a> |
  <a routerLink="/about">About</a>
</nav>

<!-- This will display the routed component -->
<router-outlet></router-outlet>
```

📌 **routerLink**: Defines clickable navigation links.

📌 **<router-outlet>**: A placeholder that dynamically loads the routed component.

Redirects and Wildcard Routes

📌 **Redirecting empty paths:**

```
{ path: '', redirectTo: '/home', pathMatch: 'full' }
```

📌 **Handling unknown routes (404 page):**

```
{ path: '**', component: PageNotFoundComponent }
```

Update **app-routing.module.ts**:

```
const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '**', component: PageNotFoundComponent }
];
```

Passing Data via Routes (Route Parameters)

You can pass dynamic data in routes using **route parameters**.

Modify **app-routing.module.ts**:

```
const routes: Routes = [
  { path: 'user/:id', component: UserComponent }
```

```
];
```

Create the UserComponent:

```
ng g c user
```

Modify **user.component.ts**:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-user',
  templateUrl: './user.component.html',
  styleUrls: ['./user.component.css']
})
export class UserComponent implements OnInit {
  userId: string | null = '';

  constructor(private route: ActivatedRoute) {}

  ngOnInit() {
    this.userId = this.route.snapshot.paramMap.get('id');
  }
}
```

Modify **user.component.html**:

```
<h2>User ID: {{ userId }}</h2>
```

Now, navigate to http://localhost:4200/user/123, and it will display:

User ID: 123.

Angular Animations

Angular provides a built-in **animations module** that allows you to create smooth, dynamic transitions for elements.

Setting Up Angular Animations

Angular animations are built using the **@angular/animations** package.

Import BrowserAnimationsModule in app.module.ts

Basic Animations

Angular animations are defined using **trigger, state, style, transition, and animate**.

Modify **app.component.ts**:

```
import { Component } from '@angular/core';
import { trigger, state, style, transition, animate } from '@angular/animations';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  animations: [
    trigger('fade', [
      state('void', style({ opacity: 0 })), // Before element appears
      transition(':enter, :leave', [animate('0.5s ease-in-out')]) // Enter & Leave animation
    ])
  ]
})
export class AppComponent {
  showText = true;
}
```

Modify **app.component.html**:

```
<h2>Angular Animations Example</h2>

<button (click)="showText = !showText">Toggle Text</button>

<p *ngIf="showText" @fade>Welcome to Angular Animations!</p>
```

Now, when you toggle the button, the text fades in and out smoothly.

Advanced Animations

Modify **app.component.ts**:

```
animations: [
  trigger('slide', [
    transition(':enter', [
      style({ transform: 'translateX(-100%)' }),
      animate('0.5s ease-out', style({ transform: 'translateX(0%)' }))
    ]),
    transition(':leave', [
      animate('0.5s ease-in', style({ transform: 'translateX(-100%)' }))
```

```
  ])
 ])
]
```

Modify **app.component.html**:

```
<div *ngIf="showText" @slide>
  <h3>Slide Animation Example</h3>
</div>
```

This makes the text **slide in and out smoothly**.

State Management in Angular

State management is crucial for maintaining data consistency across components. There are two main approaches:

1️⃣ **Using Services & RxJS (Simple & Recommended for Small Apps)**
2️⃣ **Using NgRx (Redux-like approach for Large Apps)**

Since NgRx is used for complex applications, we'll first focus on **services with RxJS**, which is widely used and necessary for most Angular projects.

1️⃣ State Management Using Services & RxJS

The simplest way to manage state in Angular is by using **services** with **RxJS Subjects**.

🔹 Step 1: Create a Service

```
ng generate service state
```

🔹 Step 2: Implement State Management in the Service.

Modify **state.service.ts**:

```
import { Injectable } from '@angular/core';
import { BehaviorSubject } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class StateService {
  private messageSource = new BehaviorSubject<string>('Default Message');
  currentMessage = this.messageSource.asObservable();

  changeMessage(message: string) {
```

```
    this.messageSource.next(message);
  }
}
```

📌 **BehaviorSubject** stores and emits the latest state.

📌 **asObservable()** allows other components to subscribe to the state.

📌 **changeMessage()** updates the state.

🔹 Step 3: Inject the Service in Components

Modify **app.component.ts**:

```
import { Component } from '@angular/core';
import { StateService } from './state.service';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  message: string = '';

  constructor(private stateService: StateService) {}

  ngOnInit() {
    this.stateService.currentMessage.subscribe(msg => this.message = msg);
  }

  updateMessage() {
    this.stateService.changeMessage('Hello from Angular!');
  }
}
```

Modify **app.component.html**:

```
<h2>State Management with Services</h2>
<p>Message: {{ message }}</p>
<button (click)="updateMessage()">Update Message</button>
```

Now, clicking the button updates the state across the app.

Let's Deep Dive into RxJS.

What is RxJS?

RxJS is a library for handling **asynchronous data** using **observables**. It allows you to manage: ✅ API calls
✅ User input events
✅ Real-time data updates
✅ WebSocket connections

💡 Think of RxJS as a **Netflix subscription**:

- The **user (subscriber)** subscribes to a **channel (observable)**.
- The **channel (observable)** continuously sends **new movies (data)**.
- The **user (subscriber)** watches and reacts to the **new movies (data)**.

Core Building Blocks of RxJS

Concept	Description
Observable	A data source that emits values over time
Observer	Listens to the observable and reacts to new values
Subscription	Connects the observer to an observable
Operators	Functions that transform data streams
Subject	A special type of observable that allows multiple subscribers

Creating & Subscribing to Observables

🔹 Example: Basic Observable

Modify **app.component.ts**:

```
import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent implements OnInit {
  data$: Observable<number>;

  ngOnInit() {
    this.data$ = new Observable(observer => {
      let count = 0;
```

```
    setInterval(() => {
      observer.next(count++); // Emits a new value every second
    }, 1000);
  });

  this.data$.subscribe(value => {
    console.log('Received:', value);
  });
 }
}
```

This creates an observable that **emits numbers every second**.

Unsubscribing to Prevent Memory Leaks

If you don't unsubscribe, observables **keep running**, causing memory leaks.

```
import { Subscription } from 'rxjs';

export class AppComponent implements OnInit {
 data$: Observable<number>;
 subscription: Subscription;

 ngOnInit() {
  this.data$ = new Observable(observer => {
    let count = 0;
    setInterval(() => {
      observer.next(count++);
    }, 1000);
  });

  this.subscription = this.data$.subscribe(value => console.log('Received:', value));
 }

 ngOnDestroy() {
  this.subscription.unsubscribe(); // Stops the observable when the component is destroyed
 }
}
```

Now, the observable stops when the component is destroyed.

RxJS Operators (The Most Powerful Part)

RxJS operators transform data **before it reaches subscribers**.

Operator	Purpose
map	Transforms each emitted value
filter	Filters values based on a condition
debounceTime	Delays emissions to reduce rapid inputs (useful for search bars)
mergeMap	Flattens and merges multiple observables
combineLatest	Combines multiple observables and emits their latest values

🔹 Example: map Operator (Transform Data)

```
import { of } from 'rxjs';
import { map } from 'rxjs/operators';

const numbers$ = of(1, 2, 3, 4, 5);
numbers$.pipe(
  map(num => num * 10) // Multiply each number by 10
).subscribe(result => console.log(result));
```

Output: 10, 20, 30, 40, 50

🔹 Example: filter Operator (Filter Values)

```
import { filter } from 'rxjs/operators';

numbers$.pipe(
  filter(num => num % 2 === 0) // Only even numbers
).subscribe(result => console.log(result));
```

Output: 2, 4

🔹 Example: debounceTime Operator (Handling Rapid Inputs)

Use this for **search bars** to avoid API spam:

```
import { fromEvent } from 'rxjs';
import { debounceTime, map } from 'rxjs/operators';

const searchBox = document.getElementById('searchBox');
const search$ = fromEvent(searchBox, 'input').pipe(
  debounceTime(500), // Wait 500ms before emitting
  map(event => (event.target as HTMLInputElement).value)
);
```

```
search$.subscribe(value => console.log('Search Input:', value));
```

Prevents unnecessary API calls while typing.

🔹 Example: mergeMap (API Call Handling)

Combines multiple API calls efficiently.

```
import { fromEvent, of } from 'rxjs';
import { mergeMap } from 'rxjs/operators';

const button = document.getElementById('myButton');
fromEvent(button, 'click').pipe(
  mergeMap(() => of('API Call Started'))
).subscribe(response => console.log(response));
```

Executes an API call every time the button is clicked.

Subjects (Special Observables)

Type	Description
Subject	A normal observable with multiple subscribers
BehaviorSubject	Stores the last emitted value
ReplaySubject	Stores multiple previous values
AsyncSubject	Emits only the last value when completed

🔹 Example: BehaviorSubject (Stores Last Value)

```
import { BehaviorSubject } from 'rxjs';

const subject = new BehaviorSubject('Initial Value');
subject.subscribe(value => console.log('Subscriber 1:', value));

subject.next('Updated Value'); // Emits to all subscribers
subject.subscribe(value => console.log('Subscriber 2:', value));
```

```
Output:
Subscriber 1: Initial Value
Subscriber 1: Updated Value
Subscriber 2: Updated Value
```